ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

ORGANISATION OF ISLAMIC COOPERATION (OIC)

Department of Computer Science and Engineering (CSE)

SEMESTER FINAL EXAMINATION — WINTER SEMESTER, 2023-2024

DURATION: 2 HOURS — FULL MARKS: 120

CSE 4107: Structured Programming I

Programmable calculators are not allowed. Do not write anything on the question paper.

Answer **all 4 (four)** questions. Figures in the right margin indicate full marks of questions with corresponding COs and POs in parentheses.

---

1. a) Explain how recursive functions work and discuss a potential risks. 5 (CO1) (PO1)

b) Write a C program to take input a $N \times N$ integer matrix, then compute and display the sum of all elements, the maximum and minimum values, and print the minor diagonal. A sample input and output for the program is shown in Table 1. 15 (CO3) (PO2)

Note: Minor diagonal of a matrix runs from top-right corner to the bottom-left corner.

Table 1: Sample input and output for Question 1.b

Sample Input	Sample Output
Enter N: 4 Enter Matrix: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16	Sum: 136 Max: 16 Min: 1 Minor Diag: 4, 7, 10, 13

c) Write a recursive C function that reverses a given string in place without using loops. Also implement `main()` for testing the function. The function prototype: 10 (CO3) (PO2)

```
void reverseString(char str[], int left, int right)
```

Table 2: Sample input and output for Question 1.c

Sample Input	Sample Output
Enter string: Hello	Reversed string: olleH
Enter string: madam	Reversed string: madam

2. a) Declare, define, and call a function `calculateSimpleInterest()` that takes three float parameters (principal, rate, and time) and returns the simple interest using the formula: 10 (CO2) (PO1)

$$interest = \frac{principal \times rate \times time}{100}$$

Ensure proper function declaration before `main()`, definition after `main()`, and correct function calling inside `main()`.

b) Explain the concepts of pointer variables, addressing, and indirection. How do they relate, and what risks are involved in using pointers? 3 + 2 (CO2) (PO2)

c) Consider the Code Snippet 1 and answer the following: 4 + 3 (CO2) (PO2)

i. Explain the difference in scope of `staticVar`, `localVar`, and `globalVar`. Can `globalVar` be accessed from `func()`?

ii. Write the output of the program when `main()` is executed.

```
1  #include <stdio.h>
2  void func() {
3      static int staticVar = 0;
4      int localVar = 0;
5
6      staticVar++;
7      localVar++;
8
9      printf("staticVar: %d, localVar: %d\n", staticVar, localVar);
10 }
11 int globalVar = 0;
12
13 int main() {
14     func();
15     func();
16     globalVar++;
17     printf("globalVar: %d\n", globalVar);
18     return 0;
19 }
```

Code Snippet 1: A C program for Question 2.c

d) Consider the Code Snippet 2, explain how pointers are used as both function arguments and a return type. What would change if `findFirstNegative` returned `arr[i]` with return type `int` instead of `&arr[i]`. 8 (CO2) (PO2)

```
1  int* findFirstNegative(int* arr, int size) {
2      for (int i = 0; i < size; i++) {
3          if (arr[i] < 0) {
4              return &arr[i]; // Return the address of the first
                   negative number
5          }
6      }
7      return NULL; // Return NULL if no negative number is found
8  }
9
10 int main() {
11     int numbers[] = {4, -1, 7, -3, 2};
12     int* negPtr = findFirstNegative(numbers, 5);
13
14     if (negPtr) {
15         *negPtr = -10; // Modify the first negative number
16     }
17 }
```

Code Snippet 2: A C program for Question 2.d

3. a) How can you access the element `arr[2][3][4]` in a 3D array `int arr[4][5][6];` using only pointer arithmetic, i.e., without using the array subscript operator (`[]`)? 4 (CO1) (PO1)

b) State the output of the program in Code Snippet 3, and justify the answer. 6 (CO1) (PO1)

```
1  #include <stdio.h>
2  int main() {
3      int arr[] = {10, 20, 30, 40, 50, 60};
4      int *ptr1 = arr;
5      int *ptr2 = arr + 3;
6
7      printf("%d\n", *(arr + 2));
8      printf("%ld\n", ptr2 - ptr1);
9      printf("%d\n", *(ptr1 + 4));
10     printf("%d\n", ptr1[3]);
11
12     ptr1++;
13     printf("%d\n", *(ptr1 + 2));
14
15     return 0;
16 }
```

Code Snippet 3: A C program for Question 3.b

c) Write a C program to identify and process peaks in a 2D matrix `mat[row][col]` using pointer arithmetic. A peak is an element strictly greater than its possible adjacent neighbors (up, down, left, and right). Define and complete the following functions:

i. `int* findPeak(int *mat, int row, int col, int *maxNeighbour)` 10 (CO3) (PO2)
Returns the address of a peak in the matrix and sets `maxNeighbour` value with the largest possible adjacent neighbors. If no peak found, returns `NULL`.

ii. `void updatePeak(int *peak, int maxNeighbour)` 2 (CO3) (PO2)
Updates the peak with `maxNeighbour` value.

iii. In `main()` repeatedly find and update peaks until no peaks remain. 8 (CO3) (PO2)

4. a) Write a C program that processes an array of `catalog_item` structures using pointers and a `union` for efficient memory management. Complete the followings:

i. Define the Structure: 6 (CO3) (PO2)

```
Catalog_Item
 |- stock_number : int
 |- price : float
 |- item_type : enum {BOOK, MUG}
 |- item_details: union {
                    Book
                     |- title : char [50]
                     |- author: char [50]
                    Mug
                     |- design: char[50]
                     |- weight: float
                   }
```

ii. `void printCatelogItem(Catalog_Item *item)` 8 (CO3) (PO2)
Takes a pointer to `Catalog_Item` structure as argument and prints the details of the

catalog item, including the `price`, `stock_number`, `item_type` and the relevant data stored in the union based on the `item_type`.

iii. `Catalog_Item *maxPriceItem(Catalog_Item *inventory, int n)` Takes a pointer to an array of `Catalog_Item` structures and the number of items in the array. Returns a pointer to the item with the highest price. 6 (CO3) (PO2)

iv. The `main()` function to test the `printCatelogItem()`, `maxPriceItem()` function. 10 (CO3) (PO2)